

Name	AYUSHI JAPSARE
UID no.	2023301006
EXP	1

CSS	
AIM:	To implement - Caesar Cipher for both encryption and decryption of a message. - Playfair Cipher for both encryption and decryption of a message. - Hill cipher for encryption and decryption of a message.
PROBLEM STATEMENT:	Classical ciphers are the foundation of cryptography, where plaintext is transformed into ciphertext using substitution and/or transposition techniques. In this experiment, we focus on three ciphers: - Caesar Cipher – A monoalphabetic substitution cipher where each letter in the plaintext is shifted by a fixed number of positions in the alphabet. Encryption and decryption involve modular arithmetic on character positions. - Playfair Cipher – A digraph substitution cipher that encrypts pairs of letters using a 5×5 matrix generated from a keyword. Rules handle repeated letters and letters not fitting into even pairs. - Hill Cipher – A polygraphic substitution cipher that encrypts blocks of letters using matrix multiplication modulo 26. Decryption involves calculating the inverse matrix modulo 26. These techniques illustrate fundamental concepts of cryptography such as key generation, modular arithmetic, matrix operations, and letter mapping.
THEORY	Problem Definition – Classical ciphers are the foundation of cryptography, where plaintext is transformed into ciphertext using substitution and/or transposition techniques. In this experiment, we focus on three ciphers: Caesar Cipher – A monoalphabetic substitution cipher where each letter in the plaintext is shifted by a fixed number of positions in the alphabet. Encryption and decryption involve modular arithmetic on character positions.

	2. Playfair Cipher – A digraph substitution cipher that encrypts pairs of letters using a 5×5 matrix generated from a keyword. Rules handle repeated letters and letters not fitting into even pairs. 3. Hill Cipher – A polygraphic substitution cipher that encrypts blocks of letters using matrix multiplication modulo 26. Decryption involves calculating the inverse matrix modulo 26. These techniques illustrate fundamental concepts of cryptography such as key generation, modular arithmetic, matrix operations, and letter mapping.
CODE:	<pre> import numpy as np # ----- Caesar Cipher ----- def caesar_encrypt(text, shift): result = "" for char in text: if char.isalpha(): base = ord('A') if char.isupper() else ord('a') result += chr((ord(char) - base + shift) % 26 + base) else: result += char return result def caesar_decrypt(text, shift): return caesar_encrypt(text, -shift) # ----- Playfair Cipher ----- def generate_playfair_matrix(key): key = key.upper().replace("J", "I") matrix = [] for char in key: if char not in matrix and char.isalpha(): matrix.append(char) for char in "ABCDEFGHIJKLMNOPQRSTUVWXYZ": if char not in matrix: </pre>

```
        matrix.append(char)
    return [matrix[i:i+5] for i in range(0, 25, 5)]
```

```
def playfair_encrypt(text, key):
    matrix = generate_playfair_matrix(key)
    pos = {matrix[r][c]: (r, c) for r in range(5) for c in range(5)}
    text = text.upper().replace("J", "I")
    processed = []
    i = 0
    while i < len(text):
        a = text[i]
        if not a.isalpha():
            i += 1
            continue
        b = ""
        if i+1 < len(text) and text[i+1].isalpha():
            b = text[i+1]
        if a == b or b == "" or not b.isalpha():
            b = "X"
            i += 1
        else:
            i += 2
        processed.append((a, b))
    result = ""
    for a, b in processed:
        r1, c1 = pos[a]
        r2, c2 = pos[b]
        if r1 == r2:
            result += matrix[r1][(c1+1)%5] + matrix[r2][(c2+1)%5]
        elif c1 == c2:
            result += matrix[(r1+1)%5][c1] + matrix[(r2+1)%5][c2]
        else:
            result += matrix[r1][c2] + matrix[r2][c1]
    return result
```

```
def playfair_decrypt(text, key):
    matrix = generate_playfair_matrix(key)
    pos = {matrix[r][c]: (r, c) for r in range(5) for c in range(5)}
    result = ""
    for i in range(0, len(text), 2):
```

```

a, b = text[i], text[i+1]
r1, c1 = pos[a]
r2, c2 = pos[b]
if r1 == r2:
    result += matrix[r1][(c1-1)%5] + matrix[r2][(c2-1)%5]
elif c1 == c2:
    result += matrix[(r1-1)%5][c1] + matrix[(r2-1)%5][c2]
else:
    result += matrix[r1][c2] + matrix[r2][c1]
return result

# ----- Hill Cipher -----
def mod_inverse_matrix(matrix, mod=26):
    # Compute determinant
    det = int(round(np.linalg.det(matrix))) % mod
    det_inv = pow(det, -1, mod) # modular inverse of determinant

    # Compute adjugate matrix
    n = len(matrix)
    adjugate = np.zeros((n, n), dtype=int)
    for i in range(n):
        for j in range(n):
            minor = np.delete(np.delete(matrix, i, axis=0), j, axis=1)
            cofactor = ((-1) ** (i + j)) * int(round(np.linalg.det(minor)))
            adjugate[j][i] = cofactor % mod # transpose for adjugate

    return (det_inv * adjugate) % mod

def text_to_numbers(text):
    return [ord(c) - ord('A') for c in text.upper() if c.isalpha()]

def numbers_to_text(nums):
    return "".join(chr(n % 26 + ord('A')) for n in nums)

def hill_encrypt(text, key_matrix):
    text_nums = text_to_numbers(text)
    while len(text_nums) % len(key_matrix) != 0:
        text_nums.append(ord('X') - ord('A'))
    result = []

```

```

for i in range(0, len(text_nums), len(key_matrix)):
    block = np.array(text_nums[i:i+len(key_matrix)])
    enc_block = np.dot(key_matrix, block) % 26
    result.extend(enc_block)
return numbers_to_text(result)

def hill_decrypt(text, key_matrix):
    inv_matrix = mod_inverse_matrix(key_matrix)
    text_nums = text_to_numbers(text)
    result = []
    for i in range(0, len(text_nums), len(key_matrix)):
        block = np.array(text_nums[i:i+len(key_matrix)])
        dec_block = np.dot(inv_matrix, block) % 26
        result.extend(dec_block)
    return numbers_to_text(result)

# ----- Menu -----
def main():
    while True:
        print("\n=== Substitution Techniques Menu ===")
        print("1. Caesar Cipher")
        print("2. Playfair Cipher")
        print("3. Hill Cipher")
        print("4. Quit")
        choice = input("Enter choice: ")

        if choice == '1':
            text = input("Enter text: ")
            shift = int(input("Enter shift value: "))
            encrypted = caesar_encrypt(text, shift)
            decrypted = caesar_decrypt(encrypted, shift)
            print("Encrypted:", encrypted)
            print("Decrypted:", decrypted)

        elif choice == '2':
            text = input("Enter text: ")
            key = input("Enter key: ")
            encrypted = playfair_encrypt(text, key)
            decrypted = playfair_decrypt(encrypted, key)
            print("Encrypted:", encrypted)

```

```

        print("Decrypted:", decrypted)

    elif choice == '3':
        text = input("Enter text: ")
        size = int(input("Enter size of key matrix (e.g., 2 or 3): "))
        print("Enter key matrix row by row (space separated letters or
numbers):")
        key_matrix = []
        for _ in range(size):
            row_input = input().split()
            row = []
            for item in row_input:
                if item.isalpha(): # Letter input
                    row.append(ord(item.upper()) - ord('A'))
                else: # Numeric input
                    row.append(int(item))
            key_matrix.append(row)
        key_matrix = np.array(key_matrix)
        encrypted = hill_encrypt(text, key_matrix)
        decrypted = hill_decrypt(encrypted, key_matrix)
        print("Encrypted:", encrypted)
        print("Decrypted:", decrypted)

    elif choice == '4':
        break
    else:
        print("Invalid choice!")

if __name__ == "__main__":
    main()

```

OUTPUT:

Caesar cipher:

```
input
--- Substitution Techniques ---
1. Caesar Cipher
2. Playfair Cipher (Key = 'icecreame')
3. Hill Cipher
4. Quit
Enter your choice: 1
Enter the text: COOL
Enter shift value: 3
Encrypted: FRRO
Decrypted: COOL

--- Substitution Techniques ---
1. Caesar Cipher
2. Playfair Cipher (Key = 'icecreame')
3. Hill Cipher
4. Quit
Enter your choice: 1
Enter the text: CRYPTOGRAPHY
Enter shift value: 6
Encrypted: IXEVZUMXGVNE
Decrypted: CRYPTOGRAPHY
```

Playfare cipher:

```
input
--- Cipher Menu ---
1. Caesar Cipher
2. Hill Cipher (2x2)
3. Playfair Cipher
4. Quit
Enter your choice: 3
Enter plaintext: CHOCOLATE
Enter key: ICECREAME

Playfair Matrix:
I C E R A
M B D F G
H K L N O
P Q S T U
V W X Y Z
Encrypted: IKKAZLOESR

Playfair Matrix:
I C E R A
M B D F G
H K L N O
P Q S T U
V W X Y Z
Decrypted: CHOCXOLATE
```

```
input
--- Cipher Menu ---
1. Caesar Cipher
2. Hill Cipher (2x2)
3. Playfair Cipher
4. Quit
Enter your choice: 3
Enter plaintext: AISHWARYA
Enter key: MONKEY

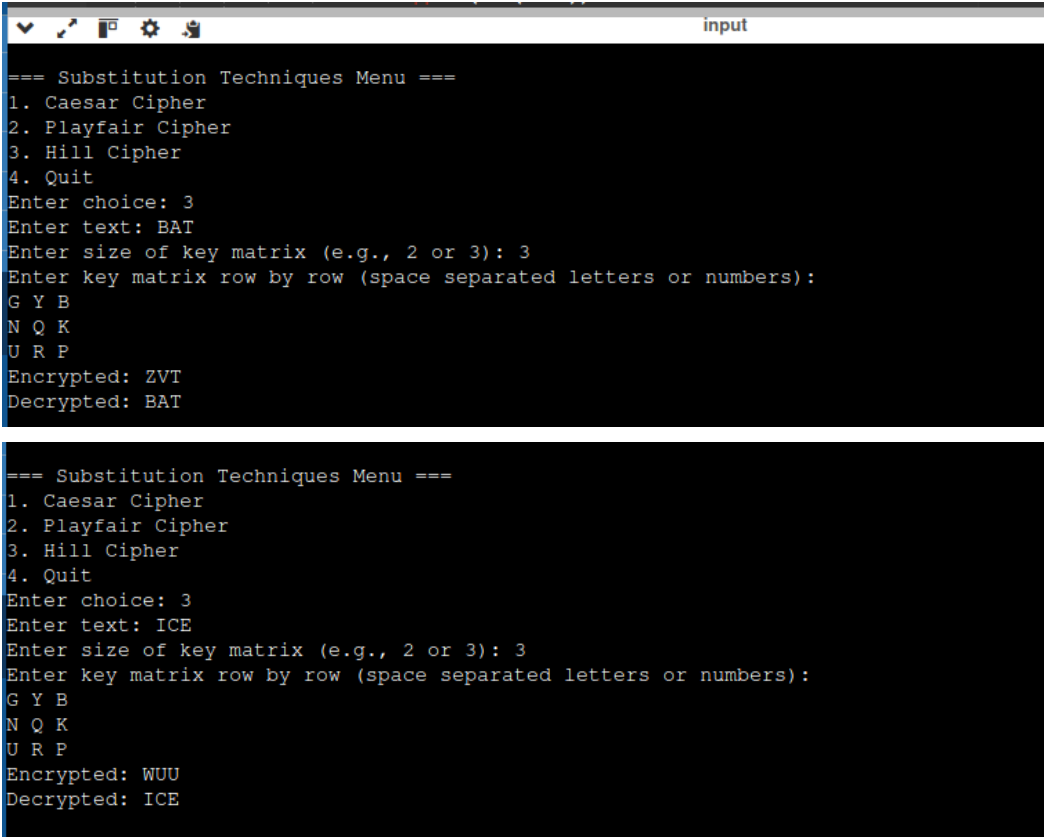
Playfair Matrix:
M O N K E
Y A B C D
F G H I L
P Q R S T
U V W X Z
Encrypted: CGRIZXBQAB

Playfair Matrix:
M O N K E
Y A B C D
F G H I L
P Q R S T
U V W X Z
Decrypted: AISHXWARYA
```

```
input
--- Cipher Menu ---
1. Caesar Cipher
2. Hill Cipher (2x2)
3. Playfair Cipher
4. Quit
Enter your choice: 3
Enter plaintext: CRYPTOGRAPHY
Enter key: MONKEY

Playfair Matrix:
M O N K E
Y A B C D
F G H I L
P Q R S T
U V W X Z
Encrypted: BSFUQEHQYQFB

Playfair Matrix:
M O N K E
Y A B C D
F G H I L
P Q R S T
U V W X Z
Decrypted: CRYPTOGRAPHY
```


	Hill cipher:  <pre> === Substitution Techniques Menu === 1. Caesar Cipher 2. Playfair Cipher 3. Hill Cipher 4. Quit Enter choice: 3 Enter text: BAT Enter size of key matrix (e.g., 2 or 3): 3 Enter key matrix row by row (space separated letters or numbers): G Y B N Q K U R P Encrypted: ZVT Decrypted: BAT === Substitution Techniques Menu === 1. Caesar Cipher 2. Playfair Cipher 3. Hill Cipher 4. Quit Enter choice: 3 Enter text: ICE Enter size of key matrix (e.g., 2 or 3): 3 Enter key matrix row by row (space separated letters or numbers): G Y B N Q K U R P Encrypted: WUU Decrypted: ICE </pre>
Conclusion:	From this experiment, I learned how classical ciphers form the basis of modern cryptography. The Caesar Cipher taught me simple substitution using modular arithmetic, while the Playfair Cipher showed how digraph substitution strengthens security. The Hill Cipher helped me understand the role of matrices and modular inverses in encryption and decryption.